

Intern Project Brief — Site Daily Report Tool

Assigned to: Intern
Duration: 2 weeks
Mentor: Check in every 2 days

What You're Building

A simple web tool where contractors fill a short daily report at the end of each working day — what work was done, how many workers were present, any blockers. A manager can then open the site's page and read all reports in one place.

No mobile app. No complex auth. Just a clean, working full-stack web app you build and deploy entirely on your own.

Tech Stack

Layer	Technology
Database	PostgreSQL (on Railway)
Backend	Node.js + Express.js (on Railway)
Frontend	HTML + CSS + vanilla JavaScript (on Vercel)

Do not use any frontend framework (no React, no Vue). Plain HTML and JS only — keep it simple.

Database Schema

Create these two tables in your PostgreSQL database:

```

CREATE TABLE sites (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  location    TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE daily_reports (
  id          SERIAL PRIMARY KEY,
  site_id     INTEGER REFERENCES sites(id),
  report_date DATE NOT NULL,
  workers_present INTEGER NOT NULL,
  work_done   TEXT NOT NULL,
  blockers    TEXT,
  submitted_by TEXT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

Backend — API Endpoints

Build these 5 endpoints in Express.js:

Sites

Method	Path	What it does
POST	/sites	Create a new site
GET	/sites	Return all sites

Reports

Method	Path	What it does
POST	/sites/:id/reports	Submit a daily report for a site
GET	/sites/:id/reports	Get all reports for a site (newest first)
GET	/sites/:id/reports/:date	Get the report for one specific date (format: YYYY-MM-DD)

Request body for POST /sites:

```
{ "name": "Prestige Sunrise", "location": "Whitefield, Bangalore" }
```

Request body for `POST /sites/:id/reports`:

```
{
  "report_date": "2026-07-08",
  "workers_present": 12,
  "work_done": "Completed plastering on floors 3 and 4. Started electrical conduit on floor 5.",
  "blockers": "Cement delivery delayed – floor 6 work on hold.",
  "submitted_by": "Rajan Kumar"
}
```

Rules:

- Only one report allowed per site per date — return a `409` error if a duplicate is submitted
- All fields except `blockers` are required — return `400` if missing
- Return proper HTTP status codes: `201` for created, `200` for fetched, `400` for bad input, `404` for not found, `409` for duplicate, `500` for server errors

Frontend — Pages

Build 3 HTML pages:

Page 1 — `index.html` — Sites List

- Shows all sites in a table or card list: name, location, date created
- Button on each site → goes to that site's reports page
- A small form at the top or bottom to add a new site (name + location)

Page 2 — `site.html?id=X` — Site Reports

- Read the site ID from the URL query param (`?id=X`)
- Show the site name at the top
- Show all daily reports in a table, newest first
- Columns: Date · Submitted By · Workers Present · Work Done · Blockers
- If no reports yet → show "No reports submitted yet"
- Button → goes to the submit form for this site

Page 3 — `submit.html?id=X` — Submit Report Form

- Read the site ID from the URL

- Form fields: Date (date picker, defaults to today), Your Name, Workers Present (number), Work Done (textarea), Blockers (textarea, optional)
 - On submit → `POST /sites/:id/reports`
 - On success → redirect to `site.html?id=X`
 - On duplicate → show error: "A report for this date already exists"
-

Deployment

Backend on Railway

1. Create a Railway account at railway.app
2. Create a new project → add a PostgreSQL database
3. Copy the `DATABASE_URL` from Railway
4. Deploy your Express.js app on Railway (connect your GitHub repo)
5. Set `DATABASE_URL` as an environment variable in Railway
6. Note down your backend's public URL (e.g. `https://your-app.up.railway.app`)

Frontend on Vercel

1. Create a Vercel account at vercel.com
2. In your frontend JS, set your Railway backend URL as a constant at the top of each file:

```
const API = 'https://your-app.up.railway.app';
```

1. Deploy your HTML files to Vercel (drag and drop or connect GitHub)
-

Project Structure

Suggested folder structure:

```
site-daily-reports/
├── backend/
│   ├── index.js          ← Express app entry point
│   ├── db.js             ← PostgreSQL connection (use 'pg' package)
│   └── routes/
│       ├── sites.js
│       └── reports.js
│   └── package.json
├── frontend/
│   ├── index.html        ← Sites list
│   ├── site.html         ← Reports for one site
│   ├── submit.html       ← Submit report form
│   └── style.css          ← Shared styles
└── README.md
```

Step-by-Step Order to Build

1. Set up Railway — create a PostgreSQL instance, run the SQL to create the two tables
2. Build the Express backend — connect to DB, write all 5 endpoints, test with Postman or curl
3. Deploy backend to Railway — confirm the endpoints work via the public URL
4. Build `index.html` — fetch sites and display, add the create site form
5. Build `submit.html` — the report submission form
6. Build `site.html` — fetch and display reports for a site
7. Deploy frontend to Vercel
8. Test the full flow end to end

Definition of Done

- ☐ Both tables created in Railway PostgreSQL
 - ☐ All 5 endpoints working and returning correct status codes
 - ☐ Duplicate report returns 409, not a server crash
 - ☐ All 3 HTML pages working and linked to each other
 - ☐ Frontend deployed on Vercel, backend on Railway
 - ☐ Full flow works: create site → submit report → view report
 - ☐ README explains how to run it locally and what each endpoint does
-

What NOT to build

- No login / authentication
 - No file uploads
 - No edit or delete (submit only)
 - No mobile app
 - No CSS framework required — keep styles minimal and clean
-

Tips

- Use the `pg` npm package to connect Express to PostgreSQL
 - Use `CORS` middleware in Express so your frontend (on Vercel) can call your backend (on Railway)
 - Test each endpoint in Postman before building the frontend
 - Commit to GitHub regularly — Railway and Vercel both deploy from GitHub
 - If stuck on SQL, `SELECT * FROM daily_reports WHERE site_id = $1 ORDER BY report_date DE`
`SC` is the main query you need
-

Useful Resources

- Express.js docs: <https://expressjs.com>
- node-postgres (pg): <https://node-postgres.com>
- Railway docs: <https://docs.railway.app>
- Vercel docs: <https://vercel.com/docs>